

Decision Log — CivicRoute Mysuru

HackMysuru 1.0 · Phase 1 · Team ZENITH (HM26-7096) · Sub-problem: Routing

Q1. What approach did we take, and what did we reject?

We built a lightweight, rule-based jurisdiction lookup: a citizen submits an area name, issue type, and description; a Flask API matches area + issue-type against a maintained lookup table and returns an authority, department, and priority before the report is even saved. We seriously considered building a true GIS approach instead: geocoding the citizen's location and running point-in-polygon lookups against ward boundary shapefiles for Mysuru City Corporation, Town Panchayats, and Gram Panchayats. That looked attractive because it is the “correct” way to resolve jurisdiction precisely, especially near contested boundary edges created by the ongoing panchayat absorption. We did not build it: sourcing accurate, up-to-date boundary shapefiles for newly absorbed panchayats within a 72-hour window was not realistic, and a wrong polygon would be worse than an honest, editable lookup table.

Q2. Why did we reject it? What was the trade-off?

We compared the two approaches on spam/edge-case resistance, offline resilience, and build complexity. The lookup table is far faster to build and easy for a non-technical admin to correct by hand when an area is mis-mapped — no GIS tooling required. Its weakness is precision: an area name is coarser than a GPS point, so a citizen literally standing on a disputed boundary street can still be routed to the wrong office if the area name itself is ambiguous. The polygon approach would have solved that precisely, at the cost of needing verified shapefiles we could not obtain in time and a materially more complex build (geocoding, spatial indexing, polygon versioning). The specific cost we knowingly accepted: jurisdiction accuracy at the boundary is only as good as how we've named and split “areas” in the lookup table, not as good as true geographic containment. We treat this as an acceptable 72-hour trade-off, not a hidden gap — it is documented in docs/limitations.md.

Q3. What breaks at the scale of all of Mysuru?

At full Mysuru scale — 65+ MCC wards plus adjacent panchayats, thousands of reports a day, and a Dasara-scale traffic spike — the first thing to break is the database layer, not the routing logic itself. SQLite is a single-writer, file-based database; under concurrent writes from many citizens reporting simultaneously, write contention will cause request timeouts well before routing logic becomes the bottleneck, and if the API ever runs on a serverless host, the SQLite file itself will not persist between invocations at all. The exact architectural change: migrate from SQLite to a hosted, connection-pooled Postgres database (Supabase or Neon both fit a fast migration since we already use SQLAlchemy, so only the connection string and a few dialect-specific queries would change), and move the area-to-authority lookup table into a properly indexed table so it can be edited by admins without a redeploy as more panchayats are absorbed.